

Лабораторная работа №5.

СУБД CLinkHouse

Цель работы. Построить ETL-конвейер для потоковой обработки данных:
генерация → брокер сообщений → аналитическая БД → визуализация.

1. Теоретические основы.

2. Порядок выполнения работы.

2.1. Общее задание.

Архитектура создаваемого конвейера

Data Generator (DataSet) → Kafka → ClickHouse (Kafka Engine) → Materialized View → MergeTree → Metabase

Используем docker.

1. На первом этапе пишем Data Generator.

Подготовим следующую структуру проектом.

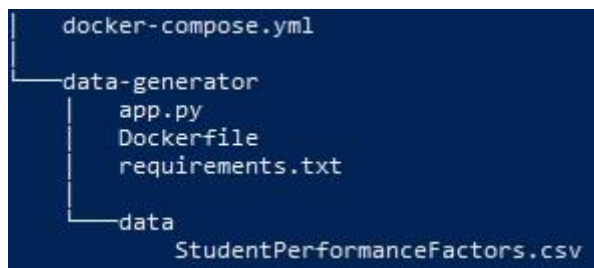


Рисунок 1 – структура проекта

В качестве датасета используем датасет из предыдущих лабораторных работ.

Содержимое docker-compose.yml

services:

data-generator:

build: ./data-generator

container_name: data-generator

environment:

DATASET_PATH: "/app/data/ StudentPerformanceFactors.csv "

EVENT_DELAY_SEC: "1.0"

LOOP_FOREVER: "false"

restart: unless-stopped

Комментарии:

services: # Раздел, в котором перечисляются все сервисы Docker Compose

data-generator: # Имя сервиса; по нему Compose управляет контейнером

build: ./data-generator # Собрать образ из Dockerfile, расположенного в папке ./data-generator

container_name: data-generator # Явно задать имя контейнера — data-generator

environment: # Переменные окружения, передаваемые внутрь контейнера

DATASET_PATH: "/app/data/ StudentPerformanceFactors.csv " # Путь к CSV-датасету внутри контейнера

EVENT_DELAY_SEC: "1.0" # Задержка между генерацией/выводом событий в секундах

LOOP_FOREVER: "false" # Не заикливать чтение датасета после достижения конца файла (или true)

restart: unless-stopped # Автоматически перезапускать контейнер, если он завершился с ошибкой

Dockerfile генератора (data-generator/Dockerfile)

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

COPY data ./data

CMD ["python", "app.py"]

Комментарии:

FROM python:3.11-slim # Использовать лёгкий базовый образ Python 3.11

WORKDIR /app # Задать рабочую директорию внутри контейнера: /app. Все последующие команды COPY, RUN, CMD будут выполняться относительно каталога /app.

COPY requirements.txt . # Скопировать файл зависимостей requirements.txt в рабочую директорию

RUN pip install --no-cache-dir -r requirements.txt # Установить Python-зависимости без сохранения кэша pip

COPY app.py . # Скопировать основной Python-скрипт генератора в контейнер

COPY data ./data # Скопировать каталог data с датасетом внутрь контейнера

CMD ["python", "app.py"] # Команда по умолчанию: запустить генератор командой python app.py

Запуск проекта.

docker compose up -d --build

docker compose - Запуск Docker Compose

up - Создать и запустить сервисы

-d - Запуск в фоновом режиме

--build - Пересобрать образ перед запуском

Пишем app.py

```
import csv
```

```
import json
```

```
import time
```

```
import logging
```

```
import os
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

```
    format="%(asctime)s - %(levelname)s - %(message)s"
```

```
)
```

```
logger = logging.getLogger(__name__)
```

```
def normalize_row(row):
```

```
    return {
```

```
        "hours_studied": int(row["Hours_Studied"]),
```

```
        "attendance": int(row["Attendance"]),
```

```
        "parental_involvement": row["Parental_Involvement"],
```

```
        "access_to_resources": row["Access_to_Resources"],
```

```
        "extracurricular_activities": row["Extracurricular_Activities"],
```

```
        "sleep_hours": int(row["Sleep_Hours"]),
```

```
        "previous_scores": int(row["Previous_Scores"]),
```

```
        "motivation_level": row["Motivation_Level"],
```

```
        "internet_access": row["Internet_Access"],
```

```
        "tutoring_sessions": int(row["Tutoring_Sessions"]),
```

```
        "family_income": row["Family_Income"],
```

```
        "teacher_quality": row["Teacher_Quality"],
```

```
        "school_type": row["School_Type"],
```

```
        "peer_influence": row["Peer_Influence"],
```

```
        "physical_activity": int(row["Physical_Activity"]),
```

```

        "learning_disabilities": row["Learning_Disabilities"],
        "parental_education_level": row["Parental_Education_Level"],
        "distance_from_home": row["Distance_from_Home"],
        "gender": row["Gender"],
        "exam_score": int(row["Exam_Score"]),
    }

```

```

def stream_dataset(file_path, delay_sec=1.0, loop_forever=False):
    sent = 0

```

```

    while True:
        logger.info(f"Streaming dataset from {file_path}")

```

```

        with open(file_path, "r", encoding="utf-8") as f:
            reader = csv.DictReader(f)

```

```

            for row in reader:
                event = normalize_row(row)
                logger.info(json.dumps(event, ensure_ascii=False))
                sent += 1

```

```

            if sent % 10 == 0:
                logger.info(f"Generated {sent} events")

```

```

            time.sleep(delay_sec)

```

```

            if not loop_forever:
                logger.info("Dataset completed, generator stopped")
                break

```

```

        logger.info("Dataset finished, restarting from beginning...")

```

```

def main():
    dataset_path = os.getenv("DATASET_PATH", "/app/data/user_events.csv")
    delay_sec = float(os.getenv("EVENT_DELAY_SEC", "1.0"))
    loop_forever = os.getenv("LOOP_FOREVER", "true").lower() == "true"

```

```

    logger.info("Starting dataset-based streaming generator")

```

```

    stream_dataset(
        file_path=dataset_path,
        delay_sec=delay_sec,
        loop_forever=loop_forever
    )

```

```

if __name__ == "__main__":
    main()

```

Проверяем работоспособность.

1. Проверка запуска контейнера

`docker compose ps`

2. Просмотр логов контейнера.

`docker logs -f data-generator`

```
2026-03-19 10:14:01,935 - INFO - Starting dataset-based streaming generator
2026-03-19 10:14:01,935 - INFO - Streaming dataset from /app/data/StudentPerformanceFactors.csv
2026-03-19 10:14:01,936 - INFO - {"hours_studied": 23, "attendance": 84, "parental_involvement": "Low", "access_to_resources": "High", "extracurricular_activities": "No", "sleep_hours": 7, "previous_scores": 73, "motivation_level": "Low", "internet_access": "Yes", "tutoring_sessions": 0, "family_income": "Low", "teacher_quality": "Medium", "school_type": "Public", "peer_influence": "Positive", "physical_activity": 3, "learning_disabilities": "No", "parental_education_level": "High School", "distance_from_home": "Near", "gender": "Male", "exam_score": 67}
2026-03-19 10:14:02,936 - INFO - {"hours_studied": 19, "attendance": 64, "parental_involvement": "Low", "access_to_resources": "Medium", "extracurricular_activities": "No", "sleep_hours": 8, "previous_scores": 59, "motivation_level": "Low", "internet_access": "Yes", "tutoring_sessions": 2, "family_income": "Medium", "teacher_quality": "Medium", "school_type": "Public", "peer_influence": "Negative", "physical_activity": 4, "learning_disabilities": "No", "parental_education_level": "College", "distance_from_home": "Moderate", "gender": "Female", "exam_score": 61}
2026-03-19 10:14:03,937 - INFO - {"hours_studied": 24, "attendance": 98, "parental_involvement": "Medium", "access_to_resources": "Medium", "extracurricular_activities": "Yes", "sleep_hours": 7, "previous_scores": 91, "motivation_level": "Medium", "internet_access": "Yes", "tutoring_sessions": 2, "family_income": "Medium", "teacher_quality": "Medium", "school_type": "Public", "peer_influence": "Neutral", "physical_activity": 4, "learning_disabilities": "No", "parental_education_level": "Postgraduate", "distance_from_home": "Near", "gender": "Male", "exam_score": 74}
2026-03-19 10:14:04,937 - INFO - {"hours_studied": 29, "attendance": 89, "parental_involvement": "Low", "access_to_resources": "Medium", "extracurricular_activities": "Yes", "sleep_hours": 6, "previous_scores": 85, "motivation_level": "High", "internet_access": "Yes", "tutoring_sessions": 1, "family_income": "High", "teacher_quality": "High", "school_type": "Private", "peer_influence": "Positive", "physical_activity": 5, "learning_disabilities": "No", "parental_education_level": "Postgraduate", "distance_from_home": "Far", "gender": "Female", "exam_score": 88}
```

Рисунок 2 – Сервис работает.

2/ На втором этапе добавляем Kafka и ZooKeeper.

ZooKeeper

ZooKeeper — сервис координации распределённых систем.

В связке с Kafka он используется для хранения служебной информации о брокерах, топиках, лидерах партиций и состоянии кластера.

Kafka

Kafka — распределённая платформа потоковой передачи данных.

В нашей лабораторной работе Kafka выполняет роль промежуточного буфера между генератором данных и последующими системами обработки.

Генератор отправляет события в Kafka-топик, а другие сервисы смогут их считывать независимо и асинхронно.

Новый вариант docker-compose.yml

```
services: # Список сервисов, входящих в лабораторный стенд
  zookeeper: # Сервис ZooKeeper
    image: confluentinc/cp-zookeeper:7.3.0 # Готовый Docker-образ ZooKeeper от Confluent
    container_name: zookeeper # Явное имя контейнера
    ports:
      - "2181:2181" # Проброс клиентского порта ZooKeeper на хост
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181 # Порт, на котором ZooKeeper принимает подключения клиентов
```

```

restart: unless-stopped # Перезапускать контейнер автоматически, если он
завершился с ошибкой
kafka: # Сервис Kafka broker
image: confluentinc/cp-kafka:7.3.0 # Готовый Docker-образ Kafka от Confluent
container_name: kafka # Явное имя контейнера
depends_on:
- zookeeper # Kafka запускается после ZooKeeper
ports:
- "9092:9092" # Проброс внешнего Kafka-порта на хост-машину
environment:
KAFKA_BROKER_ID: 1 # Идентификатор Kafka broker в кластере
KAFKA_ZOOKEEPER_CONNECT: "zookeeper:2181" # Адрес ZooKeeper,
используемого Kafka
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
# Карта listener-ов и используемых протоколов
KAFKA_LISTENERS:
PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST://0.0.0.0:9092
# Kafka слушает два адреса:
# 29092 — для контейнеров внутри Docker-сети
# 9092 — для подключений с хоста
KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
# Адреса, которые Kafka сообщает клиентам:
# kafka:29092 — для сервисов внутри Docker
# localhost:9092 — для локальных внешних клиентов
KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
# Listener, используемый для внутреннего взаимодействия брокеров
KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
# Автоматически создавать топики при первом обращении к ним
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
# Коэффициент репликации служебного топика offsets; для одного брокера
должен быть равен 1
restart: unless-stopped # Автоматический перезапуск Kafka при сбое
data-generator: # Генератор данных
build: ./data-generator # Сборка контейнера из локальной папки data-generator
container_name: data-generator # Явное имя контейнера
environment:
KAFKA_BROKER: "kafka:29092" # Адрес Kafka broker внутри Docker-сети
KAFKA_TOPIC: "students_events" # Имя Kafka-топика для отправки сообщений
DATASET_PATH: "/app/data/ StudentPerformanceFactors.csv" # Путь к CSV-файлу
внутри контейнера
EVENT_DELAY_SEC: "1.0" # Интервал между событиями в секундах
LOOP_FOREVER: "true" # Зацикливать чтение датасета
depends_on:
- kafka # Генератор запускается после Kafka
restart: unless-stopped # Автоматический перезапуск контейнера при сбое

```

Data-generator.

Файл data-generator/requirements.txt

kafka-python==2.0.2

Изменения в app.ru

```
import socket # Новый импорт: нужен для проверки доступности Kafka по TCP
from kafka import KafkaProducer # Новый импорт: Kafka producer для отправки
сообщений
from kafka.errors import NoBrokersAvailable # Новый импорт: обработка ошибки
отсутствия брокеров
```

```
def check_kafka_connection(host="kafka", port=29092, timeout=5):
```

```
    """
```

```
    Новая функция.
```

```
    Проверяет TCP-доступность Kafka broker.
```

```
    """
```

```
    try:
```

```
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
        sock.settimeout(timeout)
```

```
        result = sock.connect_ex((host, port))
```

```
        sock.close()
```

```
        return result == 0
```

```
    except Exception as e:
```

```
        logger.error(f"Socket error: {e}")
```

```
    return False
```

```
def wait_for_kafka(max_attempts=60, delay=2):
```

```
    """
```

```
    Новая функция.
```

```
    Ожидает, пока Kafka станет доступной.
```

```
    """
```

```
    broker = os.getenv("KAFKA_BROKER", "kafka:29092") # Новая переменная
окружения
```

```
    host, port = broker.split(":")
```

```
    port = int(port)
```

```
    logger.info(f"Waiting for Kafka at {broker}...")
```

```
    for attempt in range(max_attempts):
```

```
        if check_kafka_connection(host, port):
```

```
            logger.info(f"Kafka is available (attempt {attempt + 1})")
```

```
            return True
```

```
        logger.info(f"Kafka not available, attempt {attempt + 1}/{max_attempts}")
```

```
        time.sleep(delay)
```

```
    logger.error("Kafka is not available after all attempts")
```

```
    return False
```

```
def create_producer():
```

```
    """
```

```
    Новая функция.
```

```
    Создает объект KafkaProducer для отправки сообщений в Kafka.
```

```
    """
```

```
    bootstrap_servers = os.getenv("KAFKA_BROKER", "kafka:29092") # Чтение адреса
Kafka из переменной окружения
```

```
    logger.info(f"Connecting to Kafka: {bootstrap_servers}")
```

```
    try:
```

```
        producer = KafkaProducer(
```

```
            bootstrap_servers=bootstrap_servers,
```

```

value_serializer=lambda v: json.dumps(v, ensure_ascii=False).encode("utf-8"),
# Сериализация Python-словаря в JSON-строку и затем в bytes

retries=5,
max_block_ms=30000,
request_timeout_ms=30000,
api_version_auto_timeout_ms=30000,
)
logger.info("KafkaProducer created successfully")
return producer
except NoBrokersAvailable:
    logger.error("No Kafka brokers available")
    return None
except Exception as e:
    logger.error(f"Producer creation error: {e}")
    return None

```

stream_dataset – Функция изменена. Теперь события не просто выводятся в лог, а отправляются в Kafka. Изменения в основном цикле

```

for row in reader:
    try:
        event = normalize_row(row)
        future = producer.send(topic, value=event) # Новая строка: отправка
события в Kafka
        metadata = future.get(timeout=10) # Ожидание подтверждения от Kafka
broker

        sent += 1
        if sent % 10 == 0:
            logger.info(
                f"Sent {sent} events | "
                f"partition={metadata.partition}, offset={metadata.offset}"
            )
            # Изменённый лог: теперь показывается факт отправки в Kafka,
            # а также номер partition и offset
            time.sleep(delay_sec)
        except Exception as e:
            logger.error(f"Error sending row: {e}") # Новая обработка ошибок при
отправке
            time.sleep(2)

```

Изменения в функции main.

```

def main():
    logger.info("Starting dataset-based streaming generator")
    if not wait_for_kafka(): # Новый шаг: ожидание готовности Kafka перед началом
работы
        return
    producer = create_producer() # Новый шаг: создание Kafka producer
    if not producer:
        return
    topic = os.getenv("KAFKA_TOPIC", "students_events") # Новая переменная
окружения: имя Kafka-топика

```



```

dataset_path = os.getenv("DATASET_PATH",
"/app/data/StudentPerformanceFactors.csv")
delay_sec = float(os.getenv("EVENT_DELAY_SEC", "1.0"))
loop_forever = os.getenv("LOOP_FOREVER", "true").lower() == "true"
stream_dataset(
    producer=producer, # Новый аргумент: объект Kafka producer
    topic=topic,       # Новый аргумент: имя Kafka topic
    file_path=dataset_path,
    delay_sec=delay_sec,
    loop_forever=loop_forever
)

```

Проверка работоспособности сервисов

1. Проверка статуса контейне

```
docker compose ps
```

Ожидается, что сервисы zookeeper, kafka и data-generator имеют статус

Up

2. Проверка логов генератора

```
docker logs -f data-generator
```

Ожидаемые сообщения:

- ожидание Kafka;
- успешное подключение к Kafka;
- отправка событий;
- каждые 10 событий — сообщение о количестве отправленных

записей.

Пример:

Waiting for Kafka at kafka:29092...

Kafka is available (attempt 3)

KafkaProducer created successfully

Sent 10 events | partition=0, offset=9

3. Проверка наличия топика в Kafka

```
docker exec -it kafka kafka-topics --bootstrap-server kafka:29092 --list
```

Ожидаемый результат:

Students_events

4. Проверка наличия сообщений в Kafka

```
docker exec -it kafka kafka-console-consumer \
```

```
--bootstrap-server kafka:29092 \  
--topic students_events \  
--from-beginning \  
--max-messages 5
```

Ожидается вывод JSON-сообщений, отправленных генератором.

5. Проверка ZooKeeper

При необходимости можно посмотреть логи ZooKeeper:

```
docker logs zookeeper --tail=50
```

3. На третьем этапе добавляем ClickHouse.- таблицы Kafka

docker-compose.yml

Новый вариант docker-compose.yml (добавлен сервис ClickHouse)

```
services:  
  zookeeper:  
    image: confluentinc/cp-zookeeper:7.3.0  
    container_name: zookeeper  
    ports:  
      - "2181:2181"  
    environment:  
      ZOOKEEPER_CLIENT_PORT: 2181  
    restart: unless-stopped  
  
  kafka:  
    image: confluentinc/cp-kafka:7.3.0  
    container_name: kafka  
    depends_on:  
      - zookeeper  
    ports:  
      - "9092:9092"  
    environment:  
      KAFKA_BROKER_ID: 1  
      KAFKA_ZOOKEEPER_CONNECT: "zookeeper:2181"  
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:  
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT  
      KAFKA_LISTENERS:  
PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST://0.0.0.0:9092  
      KAFKA_ADVERTISED_LISTENERS:  
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092  
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT  
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'  
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1  
    restart: unless-stopped
```

```

clickhouse: # Новый сервис ClickHouse
  image: clickhouse/clickhouse-server:24.3.3.102 # Официальный образ ClickHouse
  Server
  container_name: clickhouse # Явное имя контейнера
  ports:
    - "8123:8123" # HTTP-интерфейс ClickHouse
    - "9000:9000" # Нативный TCP-интерфейс ClickHouse
  environment:
    CLICKHOUSE_DB: default # База данных по умолчанию
    CLICKHOUSE_USER: default # Пользователь по умолчанию
    CLICKHOUSE_PASSWORD: "" # Пустой пароль для локального учебного
стенда
  volumes: # Подключаются постоянное хранилище данных ClickHouse и SQL-
скрипт начальной инициализации базы.
    - ./clickhouse/init.sql:/docker-entrypoint-initdb.d/init.sql # SQL-скрипт
автоматической инициализации ClickHouse
    - clickhouse_data:/var/lib/clickhouse # Docker volume для хранения данных
ClickHouse
  depends_on:
    - kafka # ClickHouse запускается после Kafka, так как далее будет читать
данные из Kafka
  restart: unless-stopped # Автоматический перезапуск контейнера при сбое

data-generator:
  build: ./data-generator
  container_name: data-generator
  environment:
    KAFKA_BROKER: "kafka:29092"
    KAFKA_TOPIC: "students_events"
    DATASET_PATH: "/app/data/StudentPerformanceFactors.csv"
    EVENT_DELAY_SEC: "1.0"
    LOOP_FOREVER: "true"
  depends_on:
    - kafka
  restart: unless-stopped

```

```

volumes:
  clickhouse_data: # Новый именованный том для постоянного хранения данных

```

Изменяем структуру проекта.

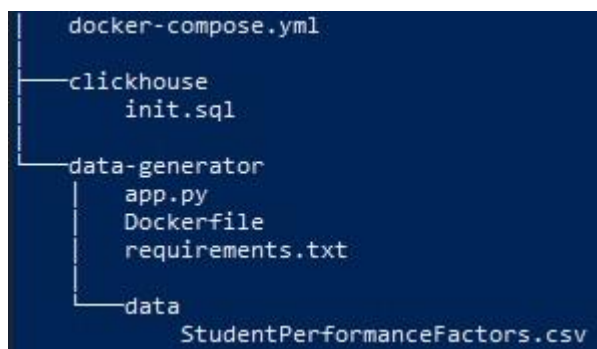


Рисунок 2 – Структура проекта

Для сервиса ClickHouse был создан каталог clickhouse и пустой файл init.sql, который будет использоваться на следующих этапах для автоматического создания таблиц и начальной настройки базы данных.

Содержимое init.sql (создаем две таблицы аналогии с DWH, но без отдельных таблиц измерений)

```
CREATE TABLE IF NOT EXISTS students_studies_kafka
(
    hours_studied UInt16,
    attendance UInt8,
    parental_involvement String,
    access_to_resources String,
    extracurricular_activities String,
    sleep_hours UInt8,
    previous_scores UInt8,
    motivation_level String,
    internet_access String,
    tutoring_sessions UInt8,
    family_income String,
    teacher_quality String,
    school_type String,
    peer_influence String,
    physical_activity UInt8,
    learning_disabilities String,
    parental_education_level String,
    distance_from_home String,
    gender String,
    exam_score UInt8
)
ENGINE = Kafka
SETTINGS
    kafka_broker_list = 'kafka:29092',
    kafka_topic_list = 'students_events',
    kafka_group_name = 'clickhouse_students_consumer_v1',
    kafka_format = 'JSONEachRow',
    kafka_num_consumers = 1,
    kafka_skip_broken_messages = 100;
```

Комментарии

1. kafka_broker_list = 'kafka:29092'

Указывает адрес Kafka broker, к которому ClickHouse будет подключаться для чтения сообщений.

kafka — имя сервиса Kafka внутри Docker-сети;

29092 — внутренний порт Kafka для контейнеров.

Эта настройка определяет, откуда ClickHouse получает поток данных.

2. `kafka_topic_list = 'students_events'`

Определяет имя Kafka-топика, из которого будет читаться поток сообщений.

В данном случае ClickHouse считывает события из топика `students_events`, в который данные отправляет сервис `data-generator`.

3. `kafka_group_name = 'clickhouse_students_consumer_v1'`

Задаёт имя consumer group, от имени которой ClickHouse подключается к Kafka.

Consumer group позволяет Kafka отслеживать:

- какие сообщения уже были прочитаны;
- с какого offset нужно продолжать чтение.

Использование отдельного имени группы позволяет ClickHouse выступать как самостоятельный потребитель потока данных.

4. `kafka_format = 'JSONEachRow'`

Определяет формат входящих сообщений.

`JSONEachRow` означает, что каждое Kafka-сообщение должно содержать одну JSON-запись, поля которой соответствуют структуре таблицы.

Пример сообщения:

JSON

```
{  
  "hours_studied": 10,  
  "attendance": 85,  
  "parental_involvement": "High",  
  "access_to_resources": "Medium",  
  ...  
}
```

ClickHouse будет автоматически сопоставлять поля JSON с колонками таблицы.

5. `kafka_num_consumers = 1`

Задаёт количество параллельных consumer-потоков, которые ClickHouse использует для чтения Kafka-топика.

Значение 1 означает:

- используется один consumer;
- чтение выполняется последовательно;
- этого достаточно для учебного стенда и небольших объёмов данных.

В промышленной среде значение может быть увеличено для повышения производительности.

6. `kafka_skip_broken_messages = 100`

Определяет, сколько некорректных сообщений ClickHouse может пропустить, не останавливая обработку потока.

Если сообщение:

- не соответствует формату JSON;
- содержит ошибки;
- не совпадает со схемой таблицы,

то ClickHouse пропустит его.

Значение 100 означает, что можно пропустить до 100 ошибочных сообщений.

Эта настройка повышает устойчивость лабораторного стенда к отдельным ошибкам во входных данных.

7. С точки зрения текущего генератора таблица согласована корректно.

Но если:

`attendance > 255,`

`previous_scores > 255,`

`sleep_hours > 255,`

`physical_activity > 255,`

то тип `UInt8` перестанет подходить для хранения.

8. На этапе создания Kafka-таблицы категориальные поля оставлены типом `String`, поскольку таблица используется как потоковый источник данных. Это упрощает чтение JSON-сообщений из Kafka и снижает вероятность ошибок при загрузке. Оптимизация до `LowCardinality(String)` будет выполнена позже в таблицах хранения `MergeTree`.

Для Kafka-таблицы `ORDER BY` не задаётся, поскольку таблицы с `ENGINE = Kafka` не хранят данные на диске как обычные таблицы, а работают как потоковый источник сообщений. Параметр `ORDER BY` будет использоваться на следующем этапе при создании таблицы хранения `MergeTree`. То же самое касается параметра. Для Kafka-таблицы `ORDER BY` не задаётся, поскольку таблицы с `ENGINE = Kafka` не хранят данные на диске как обычные таблицы, а работают как потоковый источник сообщений.

Параметр `ORDER BY` будет использоваться на следующем этапе при создании таблицы хранения `MergeTree`.

Перезапуск проекта при изменении init.sql

Файл `init.sql` используется для автоматической инициализации ClickHouse при первом запуске контейнера.

Важно учитывать, что SQL-скрипты из каталога `/docker-entrypoint-initdb.d/` выполняются только при первичной инициализации контейнера ClickHouse.

Это означает, что если изменить файл `init.sql` после первого запуска, новые SQL-команды автоматически применены не будут, пока не будет создан новый экземпляр хранилища ClickHouse.

Как применить изменения в init.sql

Вариант 1. Полный пересоздание контейнера ClickHouse с удалением тома данных

Если данные не нужно сохранять, можно полностью пересоздать сервис ClickHouse

```
docker compose down -v
```

```
docker compose up -d --build
```

Вариант 2. Пересоздание только ClickHouse

Если требуется заново инициализировать только ClickHouse, можно выполнить

```
docker compose stop clickhouse
```

```
docker compose rm -f clickhouse
```

```
docker volume rm <имя_тома_clickhouse>
```

```
docker compose up -d --build clickhouse
```

Имя тома можно узнать командой

```
docker volume ls
```

Если том данных ClickHouse не удалить, то при повторном запуске контейнера файл `init.sql` выполнен не будет, поскольку база уже считается инициализированной.

Проверка работоспособности.

1. Дополнительно проверяем, что таблицы созданы

```
docker exec -it clickhouse clickhouse-client --query "SHOW TABLES"
```

2. Можно посмотреть структуру таблиц.

```
docker exec -it clickhouse clickhouse-client --query "SHOW CREATE TABLE students_studies_kafka"
```

4. На четвертом этапе создаем таблицы **Materialized View** и **MergeTree**

Добавляем по следующей схеме.

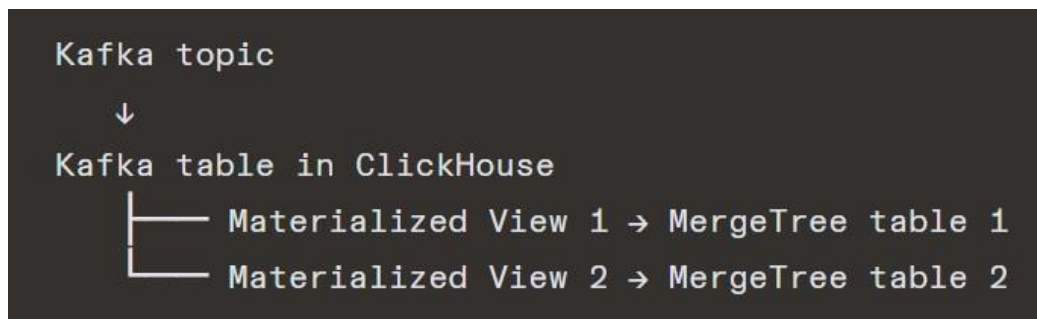


Рисунок 3 - Создание таблиц Materialized View и MergeTree

На данном этапе в ClickHouse создаются таблицы хранения типа MergeTree и материализованные представления Materialized View.

Структура таблиц MergeTree проектируется на основе двух хранилищ данных, разработанных во второй лабораторной работе. При этом в рамках текущей реализации используется упрощённый подход: вместо отдельного хранения таблицы фактов и связанных таблиц измерений соответствующие атрибуты объединяются в одну широкую таблицу MergeTree.

Такое решение позволяет:

- упростить схему хранения;
- ускорить аналитические запросы за счёт денормализации;
- сократить число соединений (JOIN) при анализе данных.

Материализованные представления Materialized View используются для автоматического переноса данных из Kafka-таблиц в таблицы MergeTree. Иными словами, Materialized View выполняют роль механизма потоковой

загрузки: при поступлении новых сообщений в Kafka-таблицу ClickHouse автоматически преобразует и записывает эти данные в целевую таблицу хранения.

4.1 Строим таблицы для первого хранилища данных (определение зависимости успеваемости от «количества занятий»).

Таблица MergeTree (код для init.sql)

```
CREATE TABLE IF NOT EXISTS students_studies_mt
(
  hours_studied UInt16,
  attendance UInt8,
  tutoring_sessions UInt8,
  exam_score UInt8,
  extracurricular_activities LowCardinality(String),
  motivation_level LowCardinality(String),
  num_studies Float32
)
ENGINE = MergeTree
ORDER BY (extracurricular_activities, motivation_level, num_studies);
```

!!! Обратите внимание, здесь используем LowCardinality и ORDER BY

Materialized View

```
CREATE MATERIALIZED VIEW IF NOT EXISTS students_studies_mv
TO students_studies_mt
AS
SELECT
  hours_studied,
  attendance,
  tutoring_sessions,
  exam_score,
  extracurricular_activities,
  motivation_level,

  0.8 * (hours_studied / 50.0)
  + 1.0 * (attendance / 100.0)
  + 0.6 * (tutoring_sessions / 8.0) AS num_studies

FROM students_studies_kafka;
```

Исходная формула

$$\begin{aligned} \text{num_studies} = & a * (\text{hours_studied} / \text{max_hours_studied}) \\ & + b * (\text{attendance} / 100) \\ & + c * (\text{tutoring_sessions} / \text{max_tutoring_sessions}) \end{aligned}$$

требует рассчитанных значений `max_hours_studied` и `max_tutoring_sessions`.

Но в потоковом режиме данные приходят постепенно, не известны заранее полностью, максимум может меняться со временем. Если использовать текущий максимум потока, то одно и то же значение будет нормализоваться по-разному в разные моменты времени. Вместо этого:

- используют фиксированные константы;
- используют заранее известные бизнес-границы (практически то же самое, что фиксированные константы, но обосновано предметной областью);
- считают агрегаты отдельно (сырые значения в MergeTree, максимумы в отдельной агрегатной таблице, нормализация делается уже в аналитическом запросе).

В данной лабораторной работе для вычисления показателя `num_studies` используются фиксированные нормализующие константы

Перезапуск init.sql

Так как `init.sql` выполняется только при первичной инициализации ClickHouse, при его изменении нужно удалить старый volume ClickHouse и поднять стенд заново.

```
docker compose down -v
```

```
docker volume ls
```

```
docker compose up -d --build
```

Если старый volume не удалится автоматически, удалить его вручную/

```
docker volume rm clickhouse-lab_clickhouse_data
```

```
docker compose up -d --build
```

Для отладки намного удобнее не пересоздавать весь стенд, а удалять и создавать таблицы вручную через clickhouse-client

Если есть связка:

Kafka table → Materialized View → MergeTree table

то удалять нужно в правильном порядке:

1. Materialized View

2. Kafka-таблица

3. MergeTree-таблица

Запросы на удаление таблиц

```
DROP VIEW IF EXISTS students_studies_mv;
```

```
DROP TABLE IF EXISTS students_studies_kafka;
```

```
DROP TABLE IF EXISTS students_studies_mt;
```

Запросы на создание приведены ранее.

Выполнить запросы можно:

1. Вручную. Войти в ClickHouse

```
docker exec -it clickhouse clickhouse-client
```

Выполнить запросы.

2. Выполнить из файла

```
docker exec -i clickhouse clickhouse-client < debug.sql
```

```
docker exec -i clickhouse clickhouse-client --multiquery < debug.sql
```

или

```
cat debug.sql | docker exec -i clickhouse clickhouse-client --multiquery
```

Может понадобиться преобразовать файл, если он редактировался в Windows - dos2unix (установка `apt install dos2unix`).

Проверка работоспособности.

1. Смотрим перечень таблиц.

```
docker exec -it clickhouse clickhouse-client --query "SHOW TABLES"
```

Ожидаемый результат:

- students_studies_kafka

- students_studies_mt

- students_studies_mv

2. Проверка структуры таблиц

```
docker exec -it clickhouse clickhouse-client --query "SHOW CREATE TABLE students_studies_kafka"
```

```
docker exec -it clickhouse clickhouse-client --query "SHOW CREATE TABLE students_studies_mt"
```

```
docker exec -it clickhouse clickhouse-client --query "SHOW CREATE TABLE students_studies_mv"
```

3. Проверка количества строк в таблице MergeTree

```
docker exec -it clickhouse clickhouse-client --query "SELECT count() FROM students_studies_mt"
```

Ожидаемый результат. Если поток данных работает корректно, результат должен быть больше нуля и увеличиваться по мере работы генератора.

4. Проверка того, что Materialized View записывает данные в MergeTree Материализованное представление напрямую читаться не должно, однако факт его работы можно проверить косвенно — по появлению строк в таблице `students_studies_mt`

```
docker exec -it clickhouse clickhouse-client --query "  
SELECT  
  hours_studied,  
  attendance,  
  tutoring_sessions,  
  exam_score,  
  extracurricular_activities,  
  motivation_level,  
  num_studies  
FROM students_studies_mt  
LIMIT 10  
"
```

Обратите внимание на вычисляемое поле — должно вычисляться (не нули).

4.2 Строим таблицы для первого хранилища данных (определение зависимости успеваемости от «домашних факторов»).

Таблица MergeTree (код для `init.sql`)

```
CREATE TABLE IF NOT EXISTS students_home_factors_mt  
(  
  parental_involvement LowCardinality(String),  
  internet_access LowCardinality(String),  
  family_income LowCardinality(String),  
  parental_education_level LowCardinality(String),  
  distance_from_home LowCardinality(String),  
  sleep_hours_group LowCardinality(String),  
  exam_score UInt8  
)  
ENGINE = MergeTree
```

```

ORDER BY
(
    parental_involvement,
    internet_access,
    family_income,
    parental_education_level,
    distance_from_home,
    sleep_hours_group
);
Materialized View

CREATE MATERIALIZED VIEW IF NOT EXISTS students_home_factors_mv
TO students_home_factors_mt
AS
SELECT
    parental_involvement,
    internet_access,
    family_income,
    parental_education_level,
    distance_from_home,

    multiIf(
        sleep_hours < 4, '0-4 hours',
        sleep_hours < 8, '4-7 hours',
        sleep_hours < 12, '8-11 hours',
        '12+ hours'
    ) AS sleep_hours_group,

    exam_score
FROM students_studies_kafka;

```

Обратите внимание на признак `sleep_hours`. Во второй лабораторной работе для него использовалось отдельное измерение и вспомогательная таблица метаданных. В ClickHouse на данном этапе преобразование выполняется непосредственно в Materialized View: числовое значение `sleep_hours` преобразуется в интервальную категорию `sleep_hours_group`.

Проверка работоспособности производится аналогично предыдущей связке MergeTree – MV.

5. Подключение Metabase

Metabase представляет собой BI-инструмент для визуализации и анализа данных. Он предоставляет веб-интерфейс, позволяющий:

- подключаться к базам данных;
- выполнять SQL-запросы;

- строить таблицы, графики и диаграммы;
- создавать интерактивные дашборды.

В рамках лабораторной работы Metabase используется как средство визуального анализа данных, загруженных в ClickHouse.

На предыдущих этапах была построена цепочка:

data-generator → Kafka → ClickHouse Kafka table → Materialized View
→ MergeTree

На пятом этапе к этой архитектуре добавляется Metabase, который подключается к ClickHouse и позволяет исследовать накопленные данные через графический интерфейс.

Итоговая схема становится следующей:

data-generator → Kafka → ClickHouse → Metabase

Изменение docker-compose.yml

Для подключения Metabase в Docker Compose добавляется новый сервис metabase

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.3.0
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    restart: unless-stopped

  kafka:
    image: confluentinc/cp-kafka:7.3.0
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: "zookeeper:2181"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_LISTENERS:
PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
```

KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
restart: unless-stopped

clickhouse:

image: clickhouse/clickhouse-server:24.3.3.102
container_name: clickhouse
ports:
- "8123:8123"
- "9000:9000"
environment:
CLICKHOUSE_DB: default
CLICKHOUSE_USER: default
CLICKHOUSE_PASSWORD: ""
volumes:
- ./clickhouse/init.sql:/docker-entrypoint-initdb.d/init.sql
- clickhouse_data:/var/lib/clickhouse
depends_on:
- kafka
restart: unless-stopped

data-generator:

build: ./data-generator
container_name: data-generator
environment:
KAFKA_BROKER: "kafka:29092"
KAFKA_TOPIC: "students_events"
DATASET_PATH: "/app/data/StudentPerformanceFactors.csv"
EVENT_DELAY_SEC: "1.0"
LOOP_FOREVER: "true"
depends_on:
- kafka
restart: unless-stopped

metabase:

build: ./metabase
container_name: metabase
ports:
- "3000:3000"
environment:
MB_DB_TYPE: h2
MB_DB_FILE: /metabase.db/metabase.db
MB_JETTY_PORT: 3000
MB_PLUGINS_DIR: /plugins
volumes:
- metabase_data:/metabase.db
depends_on:
- clickhouse
restart: unless-stopped

volumes:

```
clickhouse_data:
metabase_data:
```

Новый сервис metabase

```
metabase: # Добавляется сервис Metabase для визуализации данных.
  build: ./metabase # Контейнер Metabase собирается из локальной папки metabase,
  в которой находится Dockerfile и ClickHouse-драйвер.
  container_name: metabase # Задаётся явное имя контейнера.
  ports:
    - "3000:3000" # Порт 3000 пробрасывается на хост-машину, чтобы веб-
  интерфейс Metabase был доступен через браузер.
  environment:
    MB_DB_TYPE: h2
    MB_DB_FILE: /metabase.db/metabase.db
    MB_JETTY_PORT: 3000
    MB_PLUGINS_DIR: /plugins
  # Назначение переменных:
  # MB_DB_TYPE: h2 — Metabase использует встроенную служебную базу H2;
  # MB_DB_FILE — путь к файлу служебной базы;
  # MB_JETTY_PORT — порт веб-сервера Metabase;
  # MB_PLUGINS_DIR — каталог, в котором находятся плагины, включая драйвер
  ClickHouse.L
  volumes:
    - metabase_data:/metabase.db #Подключается Docker volume для хранения
  служебной базы Metabase между перезапусками.
  depends_on:
    - clickhouse #Metabase запускается после ClickHouse, так как будет
  подключаться к нему как к источнику данных.
  restart: unless-stopped #Контейнер автоматически перезапускается при сбое.

volumes:
  clickhouse_data:
  metabase_data: # Новый том
```

Структура каталога metabase

Для корректной работы Metabase используется отдельный каталог

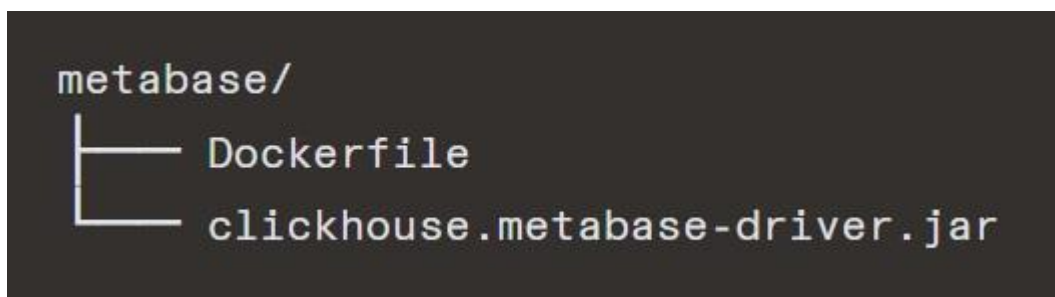


Рисунок 4 - Структура каталога metabase

Файл metabase/Dockerfile

FROM metabase/metabase:v0.48.6

USER root

RUN mkdir -p /plugins

COPY clickhouse.metabase-driver.jar /plugins/

Назначение Dockerfile:

- используется базовый образ metabase/metabase:v0.48.6;
- создаётся каталог /plugins;
- в него копируется драйвер ClickHouse, необходимый для подключения Metabase к ClickHouse.

Каталог /plugins не создаётся вручную в структуре проекта на хосте. Он создаётся внутри контейнера Metabase на этапе сборки Docker-образа командой `RUN mkdir -p /plugins`. Далее в этот каталог копируется драйвер ClickHouse, необходимый для подключения Metabase к ClickHouse.

Драйвер необходимо скачать заранее.

`wget -O clickhouse.metabase-driver.jar`
<https://github.com/ClickHouse/metabase-clickhouse-driver/releases/download/1.3.3/clickhouse.metabase-driver.jar>

Пересборка и запуск проекта

docker compose up -d --build

Проверка работоспособности

1. Проверка контейнеров

docker compose ps

В списке должен появиться контейнер metabase со статусом Up.

2. Проверка логов Metabase

`docker logs -f metabase`

Ожидаемая строка:

Metabase Initialization COMPLETE

3. Проверка веб-интерфейса

После запуска Metabase нужно открыть в браузере:

`http://localhost:3000`

Должна открыться стартовая страница Metabase.

Подключение ClickHouse в Metabase

После первого входа в Metabase необходимо:

- пройти начальную настройку;
- выбрать Add database;
- выбрать тип базы ClickHouse;
- указать параметры подключения.

Параметр	Значение
Database type	ClickHouse
Host	clickhouse
Port	8123
Database name	default
Username	default
Password	(пусто)

Проверка подключения

После сохранения подключения Metabase должен успешно подключиться к ClickHouse и отобразить доступные таблицы, например:

students_studies_mt
students_home_factors_mt

Время выполнения запросов.

1. Самый простой способ — через clickhouse-client

Если запрос выполнять в клиенте, ClickHouse обычно сам показывает время выполнения.

```
docker exec -it clickhouse clickhouse-client
```

Дальше:

```
SELECT count() FROM students_studies_mt;
```

После выполнения будет видно что-то вроде:

```
1 rows in set. Elapsed: 0.003 sec.
```

2. Через FORMAT Null для “чистого” замера

Если результат большой и не хочется тратить время на вывод:

```
SELECT *  
FROM students_studies_mt  
FORMAT Null;
```

Так измеряется почти только время выполнения запроса, без печати результата.

3. Через системный лог запросов

Можно посмотреть время уже выполненных запросов:

```
SELECT  
    query,  
    event_time,  
    query_duration_ms,  
    read_rows,  
    read_bytes,  
    result_rows  
FROM system.query_log  
WHERE type = 'QueryFinish'  
ORDER BY event_time DESC  
LIMIT 10;
```

2.2. Индивидуальное задание

1. На основе примера из общего задания разработать конвейер обработки данных для выбранного датасета

1.1 Добавьте время загрузки на этапе MV

Например, в MergeTree можно добавить колонку:

```
loaded_at DateTime DEFAULT now()
```

или в Materialized View:

```
now() AS loaded_at
```

1.2 Скорректировать значение EVENT_DELAY_SEC: "1.0" из секции data-generator файла docker-compose.yml таким образом, чтобы данные загружались за разумное время.

Оценить изменение диаграмм в metabase в режиме реального времени

1.3 Предусмотреть режим «быстрой загрузки»

Построить несколько диаграмм после полной загрузки данных и сравнить их с диаграммами, построенными для data_marts во второй лабораторной работе и аналогичными диаграммами, построенными на «сырых» данных (практическое занятие № 1).

Таким же образом сравнить диаграммы из первого практического занятия с аналогичными диаграммами, построенными на основе данных из таблиц ClickHouse.

2. Сравнить время выполнения запросов (pandas, sql dwh, clickhouse dwh).

3.